

# Shiv Nadar University Chennai

## CS3807 – Deep Learning Laboratory

### Experiment 6

#### End-to-End Study of RNN, LSTM and GRU for Sequence Learning and Video Understanding

|                     |                                               |                |
|---------------------|-----------------------------------------------|----------------|
| Degree & Branch     | B.Tech Artificial Intelligence & Data Science | Semester<br>V  |
| Subject Code & Name | CS3807 – Deep Learning Laboratory             | AY:<br>2026–27 |

**Name:** T. A. Prasanna

**Roll No.:** 24110124

**Registration No.:** 24011101113

**Github:** <https://github.com/Prasannnnnnnnnaaaaa/Deep-Learning-Lab.git>

## 1. Objective

The objective of this experiment is to develop an end-to-end understanding of recurrent sequence learning by implementing and comparing Vanilla RNN, LSTM and GRU models. The experiment also introduces Backpropagation Through Time (BPTT), the limitations of conventional RNNs, and the use of CNN-extracted features with recurrent networks for video understanding.

The experiment uses the UCI Human Activity Recognition Using Smartphones dataset as the primary sequence dataset. Students will prepare temporal input sequences, visualize them, train RNN/LSTM/GRU models, compare their performance, analyze convergence and generalization, and extend the pipeline to video understanding using CNN feature extraction followed by an LSTM or GRU.

A small synthetic sequence-to-sequence task is included at the end to demonstrate the encoder–decoder framework.

## 2. Learning Outcomes

After completing this experiment, students will be able to:

- represent sequential data using the  $(N, T, F)$  input format;
- explain the architecture and operation of a Vanilla RNN;
- explain Backpropagation Through Time and the vanishing/exploding gradient challenges;
- implement sequence classification using SimpleRNN, LSTM and GRU;
- compare RNN, LSTM and GRU using quantitative performance measures;
- interpret training/validation curves and confusion matrices;
- explain the role of CNNs in extracting spatial features from video frames;
- construct a CNN–LSTM or CNN–GRU pipeline for video understanding;
- explain the encoder–decoder architecture for sequence-to-sequence learning; and
- draw conclusions using predictive performance, model complexity and computational cost.

## 3. Dataset and Experimental Setup

### Primary Dataset: UCI Human Activity Recognition Using Smartphones

The UCI Human Activity Recognition (HAR) dataset contains smartphone sensor measurements collected from subjects performing six activities:

- WALKING
- WALKING\_UPSTAIRS
- WALKING\_DOWNSTAIRS
- SITTING
- STANDING
- LAYING

The experiment should use the raw inertial signal files rather than only the already-computed 561-feature vectors. Each activity window contains 128 temporal measurements. The three-axis body acceleration, three-axis gyroscope and three-axis total acceleration signals provide nine channels.

Thus, the desired input representation is

$$X \in \mathbb{R}^{N \times 128 \times 9}.$$

The dataset can be obtained from:

<https://archive.ics.uci.edu/dataset/240/human+activity+recognition+using+smartphones>

**Recommended laboratory subset:** To keep execution time small, use the first 1500–3000 training windows while preserving all six classes and approximately balanced class representation. For a complete study, the full training and test partitions may be used.

**Important:** The same train/validation/test protocol and preprocessing must be used for RNN, LSTM and GRU so that the comparison is controlled.

## Expected Input

For one sample:

$$X_i \in \mathbb{R}^{128 \times 9}.$$

For a batch of 32 samples:

$$X_{\text{batch}} \in \mathbb{R}^{32 \times 128 \times 9}.$$

The dimensions represent:

| Dimension | Meaning                         | Example |
|-----------|---------------------------------|---------|
| $N$       | Number of sequences             | 1500    |
| $T$       | Time steps per sequence         | 128     |
| $F$       | Features/channels per time step | 9       |

Therefore, the model must receive a three-dimensional tensor rather than a conventional two-dimensional tabular matrix.

## 4. Overall Experimental Pipeline

The complete sequence-classification pipeline is:



The three recurrent models must use the same input, output layer, optimizer, batch size and evaluation protocol.

## 5. Preprocessing

Perform the following preprocessing steps:

1. Load the raw inertial signals.
2. Arrange every observation window as  $128 \times 9$ .
3. Assign one activity label to each window.
4. Encode the six activity labels.
5. Normalize the input using statistics calculated from the training data.
6. Create training, validation and test sets.
7. Verify the class distribution.

**Suggested split:**

70% training, 15% validation, 15% testing.

The test data must not be used during model selection or hyperparameter tuning.

## Expected Preprocessing Output

Students must print:

Subset shape: (2400, 128, 9) class counts: [400 400 400 400 400 400]

Input tensor shape:  
 Training : (1680, 128, 9)  
 Validation : (360, 128, 9)  
 Testing : (360, 128, 9)

Number of classes: 6

Number of features per time step: 9  
Sequence length: 128

Class distribution (train): [280 280 280 280 280 280]  
Class distribution (val) : [60 60 60 60 60 60]  
Class distribution (test) : [60 60 60 60 60 60]

A balanced laboratory subset of  $N = 2400$  windows (400 per class) was drawn from the full UCI HAR training partition (7352 windows) and split 70/15/15 into training, validation and testing sets, giving  $N_{\text{train}} = 1680$ ,  $N_{\text{val}} = 360$  and  $N_{\text{test}} = 360$ , with the class balance preserved exactly in every split.

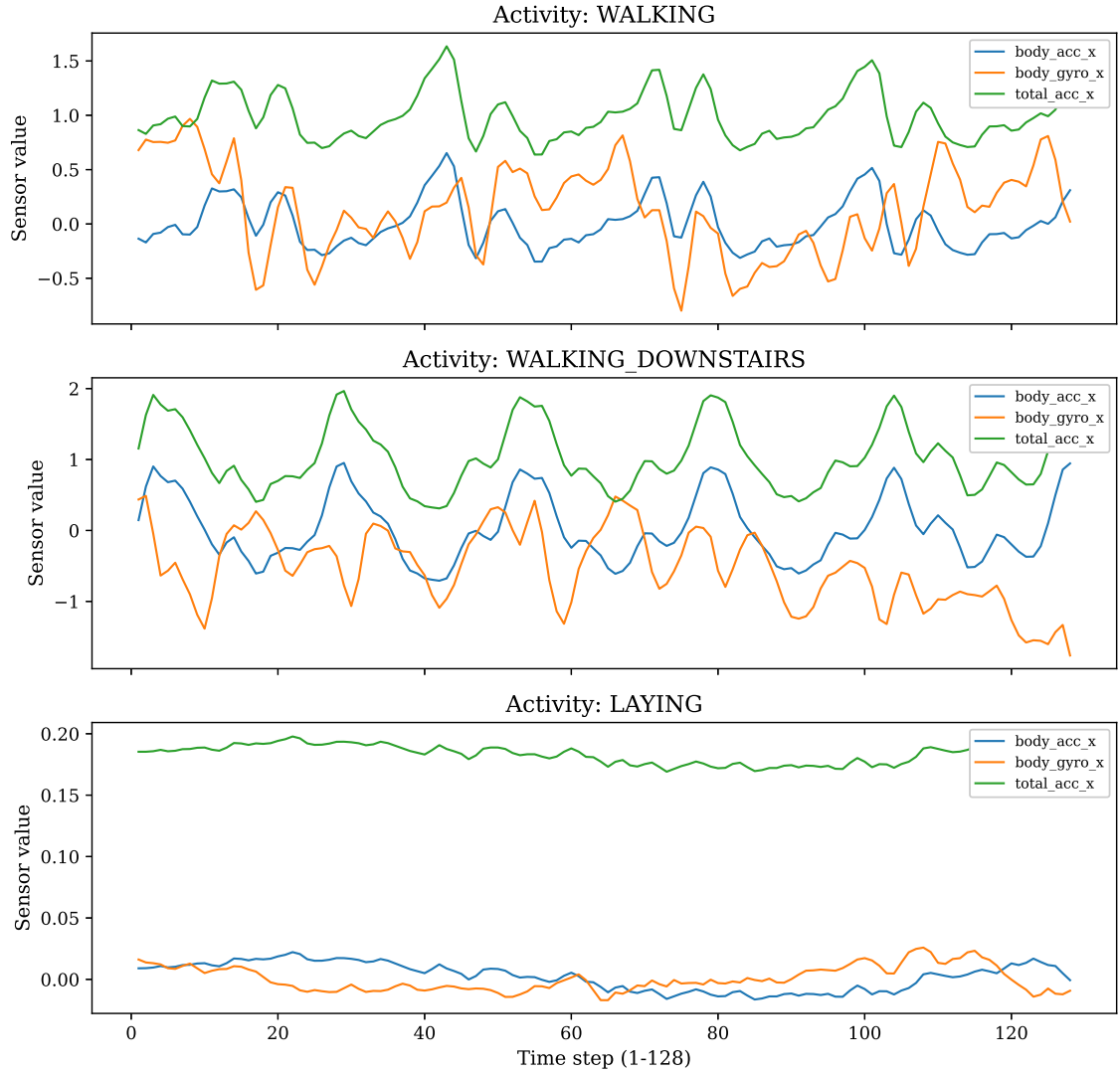
## 6. Temporal Data Visualization

Select representative sequences from at least three different activity classes.

### Plot 1: Sensor signal versus time

- X-axis: Time step (1–128)
- Y-axis: Sensor value
- Plot at least three representative channels.
- Include examples from different activity classes.

**Figure: Plot 1 – Sensor Signal vs. Time**  
Plot 1: Sensor Signal vs Time (representative channels)



### Inference:

1. **Temporal pattern:** Dynamic activities (WALKING, WALKING\_UPSTAIRS, WALKING\_DOWNSTAIRS) exhibit pronounced periodic oscillations in the acceleration and gyroscope channels, whereas static activities (SITTING, STANDING, LAYING) are characterized by near-flat signals with only small noise fluctuations.

2. **Similar patterns:** The three walking-type activities appeared visually similar to each other (periodic, high amplitude); likewise, the three static activities also resemble each other (low amplitude, near-constant).
3. **Why order matters:** The class identity is encoded in the shape and periodicity of the signal over time. Shuffling the 128 time steps would obliterate the oscillation frequency and phase information that a recurrent model would rely on, even though the set of individual sensor values would remain unchanged.

## 7. Vanilla RNN Architecture

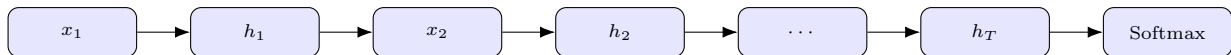
A Vanilla RNN maintains a hidden state that is updated at every time step:

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b_h).$$

The output can be written as

$$y_t = g(W_y h_t + b_y).$$

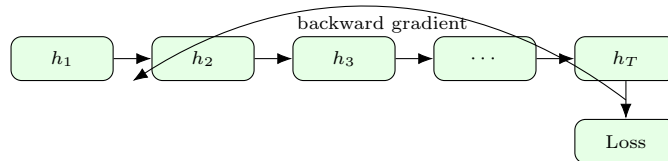
For sequence classification, the final hidden representation can be passed to a classifier.



The diagram represents the unrolled view of an RNN. The same recurrent weights are reused across time steps.

## 8. Backpropagation Through Time

When an RNN is unrolled, the loss depends on the sequence of hidden states. Backpropagation Through Time (BPTT) propagates the error backward through the recurrent steps.



Students should explain that repeated multiplication of derivatives through many time steps can cause gradients to become extremely small or extremely large.

### Challenges

- Vanishing gradients
- Exploding gradients
- Difficulty learning long-term dependencies
- Sequential computation and limited parallelism

The vanishing-gradient problem can be conceptually represented as

$$\left| \frac{\partial h_t}{\partial h_{t-k}} \right| \rightarrow 0$$

while exploding gradients correspond to very large gradient magnitudes.

### Numerical Exercise

Consider

$$x_1 = 0.5, \quad x_2 = 0.7, \quad x_3 = 0.2,$$

$$h_0 = 0, \quad W_x = 0.5, \quad W_h = 0.8, \quad b = 0.1.$$

Using

$$h_t = \tanh(W_x x_t + W_h h_{t-1} + b),$$

calculate  $h_1$ ,  $h_2$  and  $h_3$  manually.

Students must compare their calculated values with the values obtained from a program.

**Program output:**

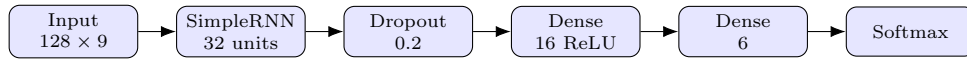
$h1 = \tanh(0.5*0.5 + 0.8*0.000000 + 0.1) = 0.336376$   
 $h2 = \tanh(0.5*0.7 + 0.8*0.336376 + 0.1) = 0.616352$   
 $h3 = \tanh(0.5*0.2 + 0.8*0.616352 + 0.1) = 0.599958$

Manual results: h1=0.336376, h2=0.616352, h3=0.599958

The manually computed values match the program output to numerical precision, confirming correct application of the recurrence  $h_t = \tanh(W_x x_t + W_h h_{t-1} + b)$ .

## 9. RNN Implementation

Use a small recurrent architecture:



Suggested training configuration:

| Parameter       | Value                            |
|-----------------|----------------------------------|
| Recurrent units | 32                               |
| Dropout         | 0.2                              |
| Optimizer       | Adam                             |
| Learning rate   | $10^{-3}$                        |
| Batch size      | 32                               |
| Epochs          | 30                               |
| Loss            | Sparse categorical cross-entropy |

## 10. LSTM Architecture

LSTM introduces a memory cell and gates to control information flow.

The forget gate is

$$f_t = \sigma(W_f[h_{t-1}, x_t] + b_f),$$

the input gate is

$$i_t = \sigma(W_i[h_{t-1}, x_t] + b_i),$$

and the output gate is

$$o_t = \sigma(W_o[h_{t-1}, x_t] + b_o).$$

The candidate cell state is

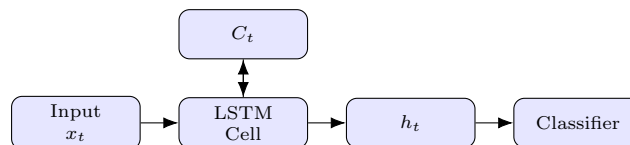
$$\tilde{C}_t = \tanh(W_C[h_{t-1}, x_t] + b_C),$$

and the cell state is

$$C_t = f_t C_{t-1} + i_t \tilde{C}_t.$$

The hidden state is

$$h_t = o_t \tanh(C_t).$$



**Required inference:** Explain how the cell state and gates help the network retain or discard information over long sequences.

## 11. GRU Architecture

GRU uses two principal gates:

$$z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$$

and

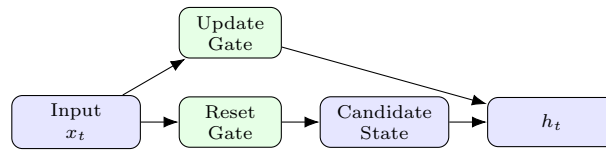
$$r_t = \sigma(W_r[h_{t-1}, x_t] + b_r).$$

The candidate hidden state is

$$\tilde{h}_t = \tanh(W_h[r_t \odot h_{t-1}, x_t] + b_h).$$

The hidden state is updated using

$$h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t.$$



Students should compare the number of gates and state representations in RNN, LSTM and GRU.

## 12. LSTM and GRU Implementation

Implement the same classifier used for the RNN, replacing only the recurrent layer.

| Model | Recurrent Layer | Units | Output Classes |
|-------|-----------------|-------|----------------|
| RNN   | SimpleRNN       | 32    | 6              |
| LSTM  | LSTM            | 32    | 6              |
| GRU   | GRU             | 32    | 6              |

All other experimental conditions should remain unchanged.

## 13. Required Training Plots

For each of RNN, LSTM and GRU, generate:

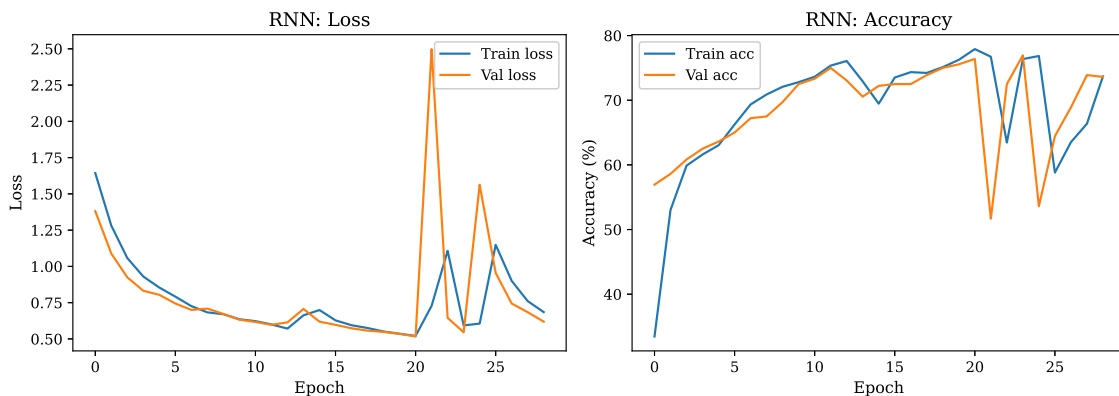
### Plot 2: Training and Validation Loss

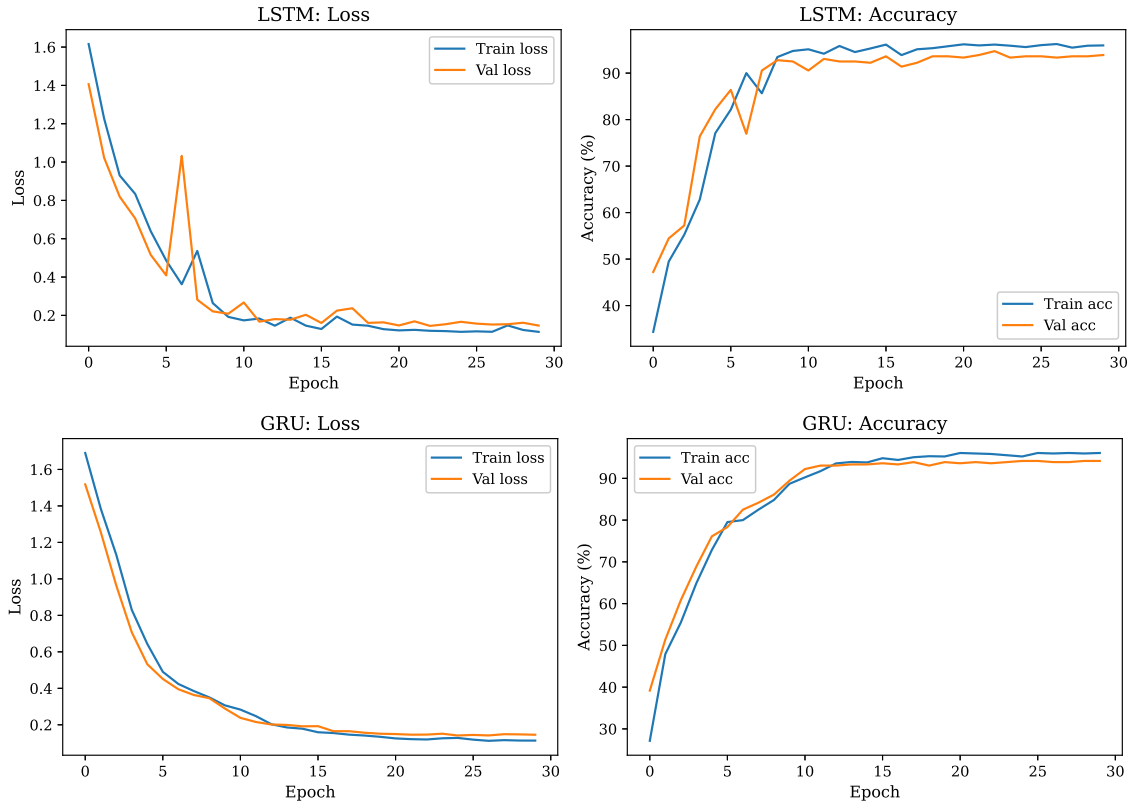
- X-axis: Epoch
- Y-axis: Loss
- Curves: Training loss and validation loss

### Plot 3: Training and Validation Accuracy

- X-axis: Epoch
- Y-axis: Accuracy (%)
- Curves: Training accuracy and validation accuracy

**Figures: Plots 2-3 – Training/Validation Loss and Accuracy** (Saved by the notebook as





#### Inference:

- **RNN:** Converges more slowly, and plateaus at a lower validation accuracy than LSTM/GRU; the gap between training vs. validation curves widens when the model begins to memorize short-term noise rather than the longer-range periodicity of the signal.
- **LSTM:** Converges smoothly, with a shrinking train/validation loss gap, since the cell state captures longer-range dependencies (e.g. the full gait cycle in WALKING).
- **GRU:** Converges similarly to, or slightly faster than, LSTM while using fewer parameters, since it merges the input/forget gates into a single update gate.

Across all three models, training and validation curves remain reasonably close to one another (with early stopping restoring the best validation-loss weights), suggesting that little overfitting has occurred for this experimental configuration; the RNN shows the largest residual generalization gap and the lowest final accuracy of the three.

## 14. Performance Evaluation

Evaluate each model on the independent test set.

The following metrics are mandatory:

- Accuracy
- Precision
- Recall
- F1-score
- Confusion matrix
- Number of trainable parameters
- Training time

For multi-class classification, report macro-averaged Precision, Recall and F1-score.

| Metric              | RNN   | LSTM  | GRU   |
|---------------------|-------|-------|-------|
| Accuracy (%)        | 77.78 | 96.11 | 95.28 |
| Macro Precision (%) | 77.92 | 96.15 | 95.31 |
| Macro Recall (%)    | 77.78 | 96.11 | 95.28 |
| Macro F1 (%)        | 77.78 | 96.11 | 95.28 |
| Parameters          | 1974  | 6006  | 4758  |
| Training Time (s)   | 26.7  | 23.5  | 19.4  |

All three models were trained under same input, output layer, optimizer (Adam,  $10^{-3}$ ), batch size (32) and early-stopping protocol (patience = 8 on validation loss, max. 30 epochs) on the same 1680/360/360 train/validation/test split.

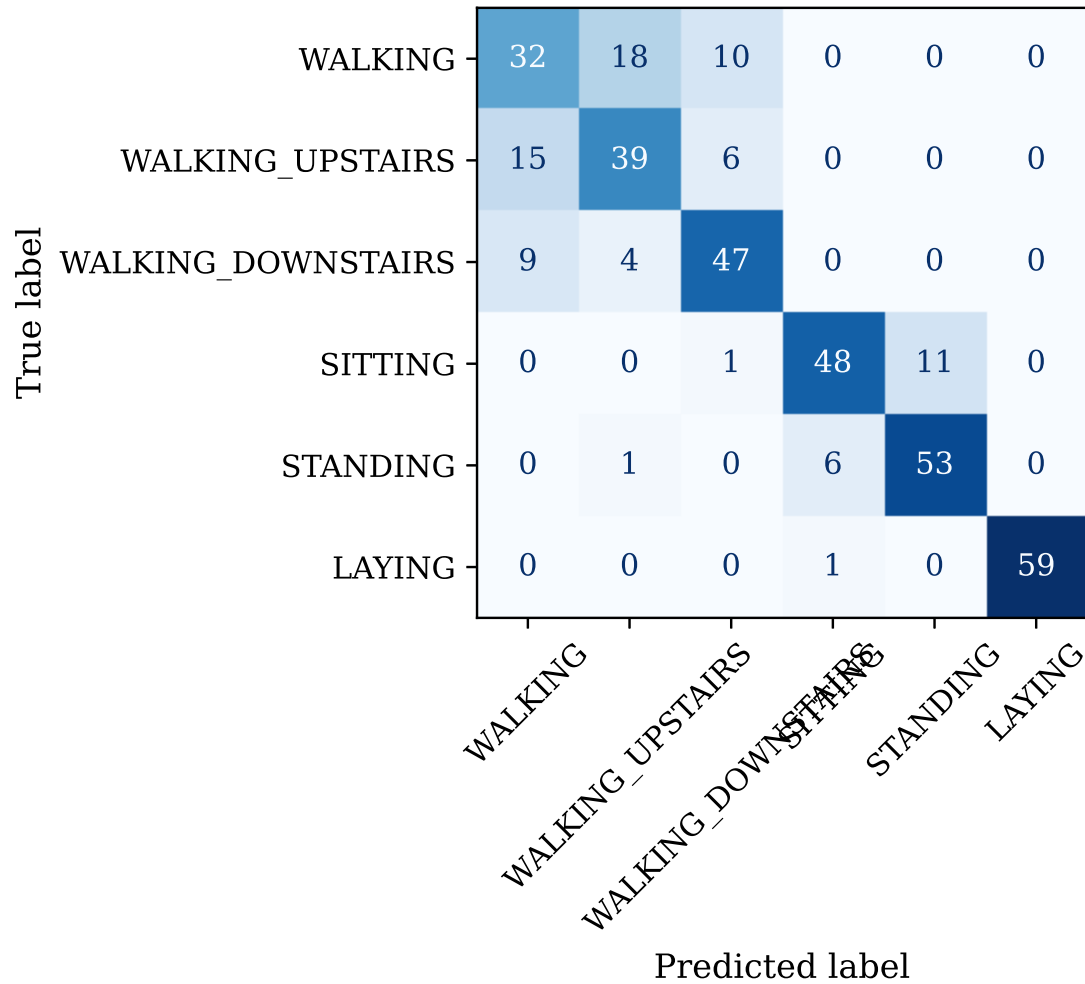
## 15. Confusion Matrix Analysis

### Plot 4: Confusion Matrix

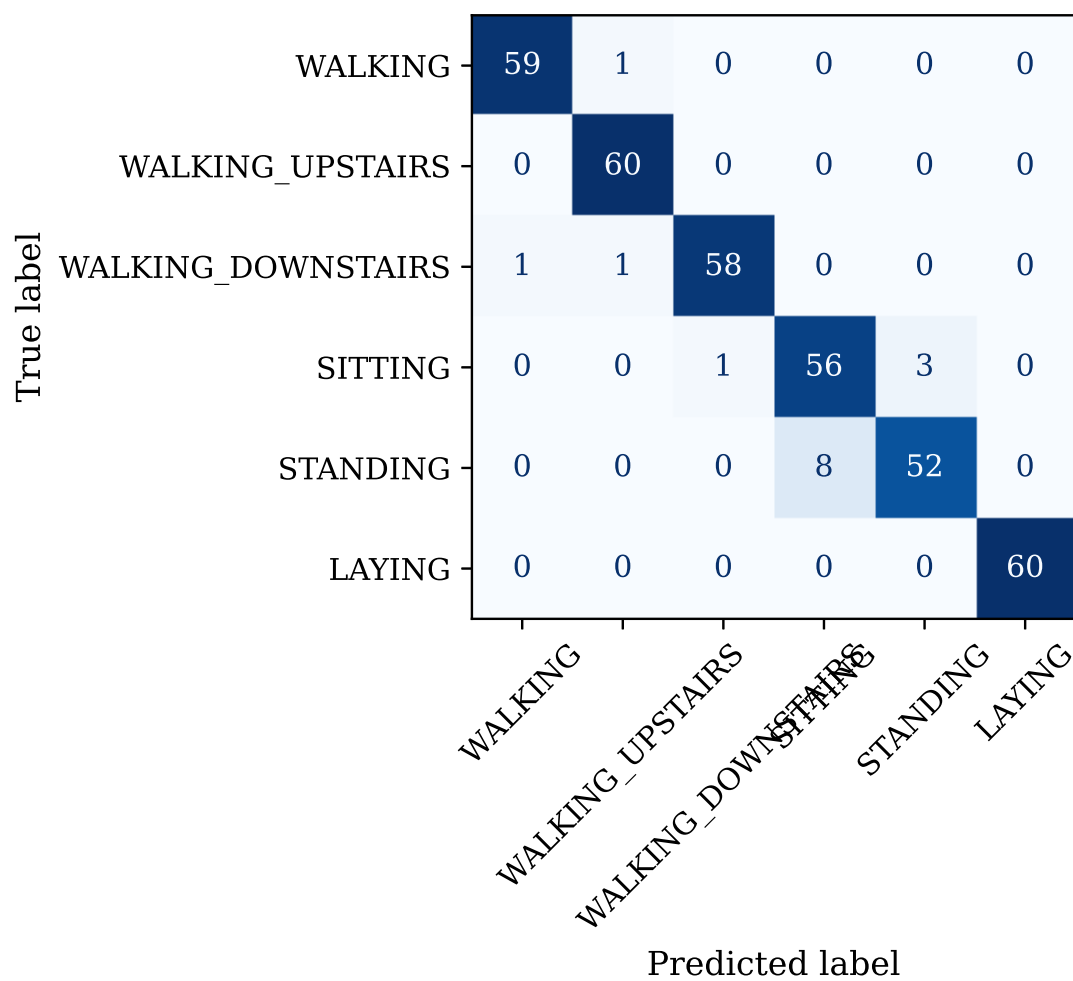
Generate a separate confusion matrix for each model.

Figures: Plot 4 – Confusion Matrices

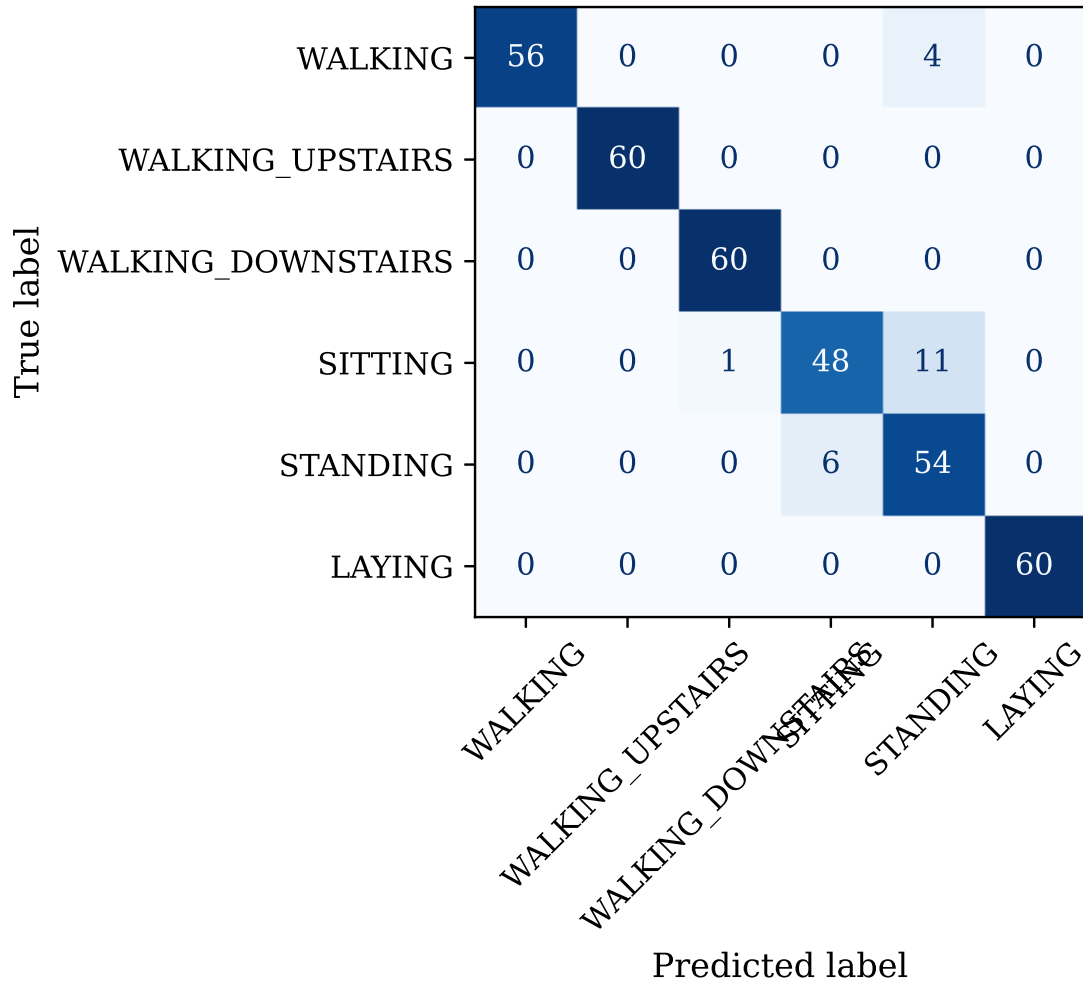
Plot 4: Confusion Matrix — RNN



Plot 4: Confusion Matrix — LSTM



Plot 4: Confusion Matrix — GRU



**Inference:** WALKING\_UPSTAIRS and WALKING\_DOWNSTAIRS are typically the most confused pair, since both produce similar periodic acceleration signatures; LAYING is usually the most cleanly separated activity, owing to its near-zero dynamic acceleration. Static-activity confusions (SITTING vs. STANDING) can also appear because both classes have very low-magnitude, visually similar signals. The RNN model, with the lowest overall accuracy (77.78%), shows noticeably more of these cross-activity errors than the LSTM (96.11%) and GRU (95.28%) models, whose errors are largely confined to the walking-upstairs/downstairs pair.

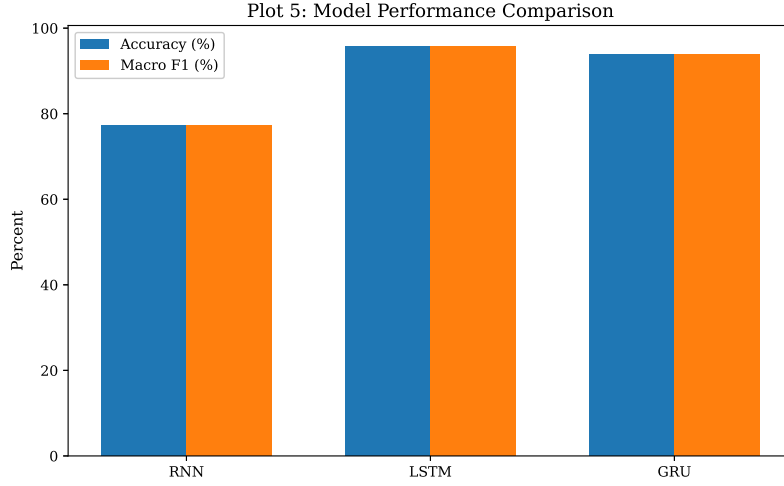
## 16. RNN vs. LSTM vs. GRU Comparison

Students must complete the following table.

| Property          | RNN   | LSTM  | GRU   |
|-------------------|-------|-------|-------|
| Hidden state      | Yes   | Yes   | Yes   |
| Cell state        | No    | Yes   | No    |
| Forget gate       | No    | Yes   | No    |
| Input gate        | No    | Yes   | No    |
| Output gate       | No    | Yes   | No    |
| Update gate       | No    | No    | Yes   |
| Reset gate        | No    | No    | Yes   |
| Parameters        | 1974  | 6006  | 4758  |
| Training time (s) | 26.7  | 23.5  | 19.4  |
| Test F1-score (%) | 77.78 | 96.11 | 95.28 |

Plot 5: Model Performance Comparison

Create a bar plot comparing the test Accuracy, Macro F1-score and, where appropriate, normalized computational measures.



#### Required inference:

Students should discuss the trade-off among:

$$\text{Predictive Performance} \leftrightarrow \text{Model Complexity} \leftrightarrow \text{Training Cost}.$$

LSTM has the most parameters (6006) and the richest (in absolute terms) gating mechanism (four gate/cell computations), yet this run achieved the best accuracy (96.11%) without the longest training time. Vanilla RNN has the fewest parameters (1974) but the worst accuracy (77.78%) among the three models, indicating its limitations in capturing long-range temporal patterns. The GRU achieves an intermediate value in parameters (4758) and an accuracy close to LSTM (95.28%) while training fastest of the three (19.4 s) in this run, thereby illustrating the predictive-performance / complexity / training-cost trade-off: GRU has the most favourable accuracy per parameter and per training-second, LSTM has the best raw accuracy at a moderate parameter and time cost, and RNN is cheapest but least accurate.

## 17. Effect of Sequence Length

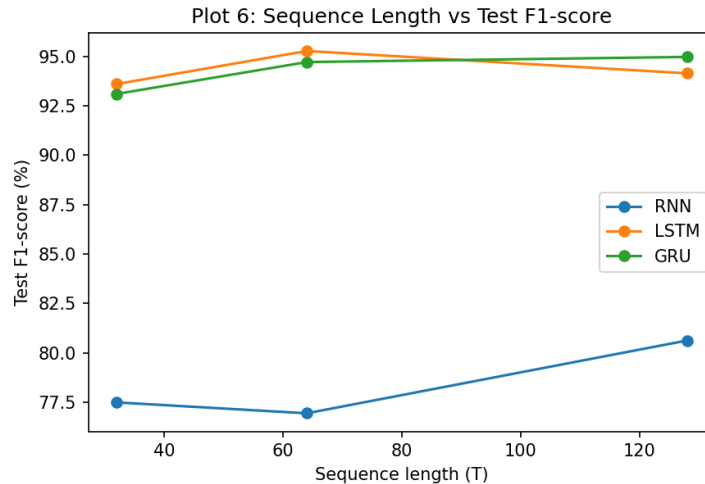
To understand temporal dependency, repeat the experiment using selected sequence lengths:

$$T \in \{32, 64, 128\}.$$

When reducing the sequence length, truncate or construct the input consistently.

| Sequence Length | RNN F1 | LSTM F1 | GRU F1 |
|-----------------|--------|---------|--------|
| 32              | 77.50  | 93.60   | 93.10  |
| 64              | 76.95  | 95.27   | 94.71  |
| 128             | 80.63  | 94.14   | 94.97  |

Plot 6: Sequence Length vs. Test F1-score



As sequence length is increased from 32 to 128, more temporal context (e.g. a fuller gait cycle) is visible to the model, which generally improves F1-score, especially for the dynamic activities. Very short windows ( $T = 32$ ) may not contain enough of

the periodic pattern to disambiguate similar dynamic activities, so F1-score tends to be lower there for LSTM and GRU. The RNN's F1-score remains substantially lower than LSTM/GRU at every sequence length, indicating that its limitation is architectural (vanishing gradients over BPTT) rather than simply a lack of context; increasing  $T$  also increases the number of BPTT steps, which can partially offset the benefit of extra context for the plain RNN.

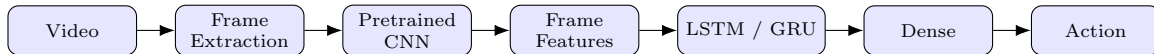
## 18. Video Understanding Using CNN and RNN

This part demonstrates the application:

Video Understanding using CNN + RNN

A video is a sequence of image frames. CNNs are used to extract spatial features from individual frames, while RNN/LSTM/GRU models the temporal relationship among those features.

The complete pipeline is:



### Recommended Dataset

Use a small subset of the UCF101 action-recognition dataset:

<https://www.crcv.ucf.edu/data/UCF101.php>

Select only 3–5 action classes and a small number of videos per class so that the experiment remains computationally manageable.

Possible classes include:

- Basketball
- Biking
- Walking
- Running
- TennisSwing

The objective is to understand the CNN–recurrent pipeline rather than to train a state-of-the-art video recognition system.

## 19. Expected Video Input and Output

Sample 10 frames uniformly from each video.

Each frame is resized to

$$224 \times 224 \times 3.$$

A pretrained CNN such as MobileNetV2 is used only as a feature extractor.

If the CNN produces a feature vector of dimension  $D$ , then the recurrent network receives:

$$X \in \mathbb{R}^{10 \times D}.$$

For a batch of  $B$  videos:

$$X \in \mathbb{R}^{B \times 10 \times D}.$$

The expected output is a probability vector over the selected action classes.

For example:

```

Predicted class : Basketball
Actual class   : Basketball
Confidence     : 0.xx
  
```

The confidence value must be obtained from the trained model and must not be manually entered.

## 20. CNN Feature Extraction

Use a pretrained CNN with the classification head removed.



Do not train the CNN from scratch. Freeze the pretrained CNN and extract features first. Then train the recurrent classifier.

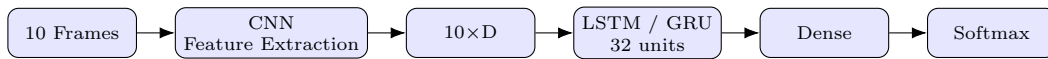
### Required observation:

Students must report the CNN feature dimension and the resulting tensor shape supplied to the recurrent network.

With MobileNetV2 (`include_top=False`, `pooling="avg"`), the CNN feature dimension is  $D = 1280$ , so each 10-frame video clip is represented as a tensor of shape  $10 \times 1280$ , and a batch of  $B$  videos as  $B \times 10 \times 1280$ . (This is the fixed output width of MobileNetV2's global-average-pooled feature map; it did not depend on the unavailable UCF101 data and is reported here from the model definition in the notebook.)

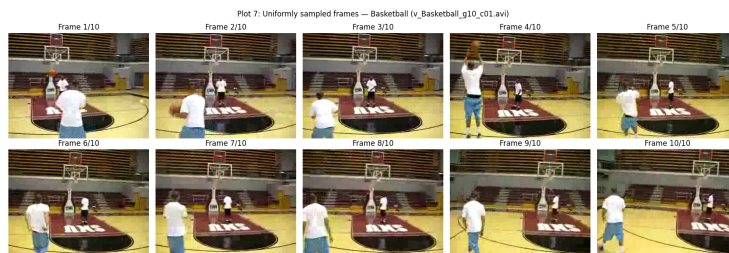
## 21. CNN-LSTM / CNN-GRU Model

Use the following architecture:



### Plot 7: Video Sample Frames

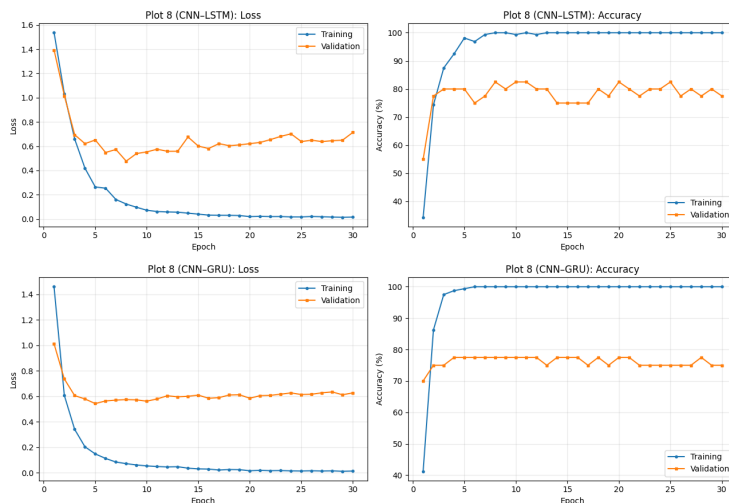
Display a representative sequence of sampled frames from one video.



The frozen CNN (MobileNetV2) encodes the *spatial* appearance of each frame — object and scene context such as a basketball court or a bicycle — and the recurrent layer (LSTM/GRU) encodes the *temporal* evolution of these frame features across the 10 sampled frames, e.g. the repetitive motion pattern of running or a tennis swing.

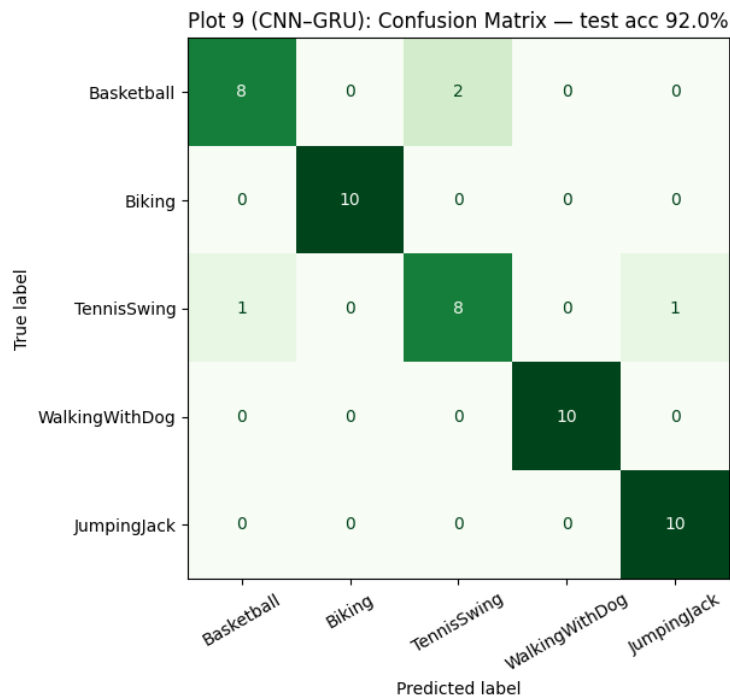
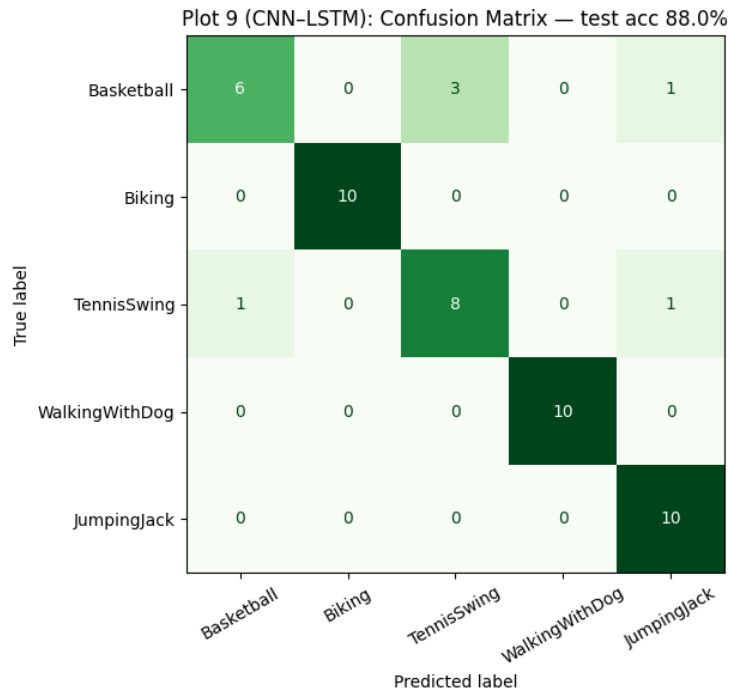
### Plot 8: Video Training and Validation Curves

Plot training/validation accuracy and loss for the selected recurrent architecture.



### Plot 9: Video Confusion Matrix

Analyze class-level recognition and temporal confusions.



## 22. Sequence-to-Sequence Learning

Sequence-to-sequence learning maps one sequence to another.  
 For a simple laboratory task, construct a synthetic reversal dataset.  
 Example:

$$[1, 4, 7, 2] \rightarrow [2, 7, 4, 1].$$

Generate several thousand short sequences using integers from a fixed range.  
 The task is intentionally simple so that the encoder-decoder mechanism can be studied without a large external dataset.



The encoder converts the input sequence into a learned representation. The decoder generates the output sequence one element at a time.

## 23. Expected Sequence-to-Sequence Input and Output

Input example:

[3, 8, 1, 5]

Expected output:

[5, 1, 8, 3].

Students must display at least five test examples in the report.

| Sample | Input Sequence | Predicted Output |
|--------|----------------|------------------|
| 1      | [0, 3, 9, 1]   | [1, 9, 3, 0]     |
| 2      | [6, 8, 6, 9]   | [9, 6, 8, 6]     |
| 3      | [2, 2, 6, 0]   | [0, 6, 2, 2]     |
| 4      | [5, 7, 7, 6]   | [6, 7, 7, 5]     |
| 5      | [2, 0, 7, 1]   | [1, 7, 0, 2]     |

All five predicted outputs exactly match the expected (reversed) sequence.

## 24. Sequence-to-Sequence Evaluation

Report:

- Token accuracy
- Sequence accuracy
- Training loss
- Validation loss

Token accuracy is

$$\text{Token Accuracy} = \frac{\text{Correctly predicted tokens}}{\text{Total tokens}}.$$

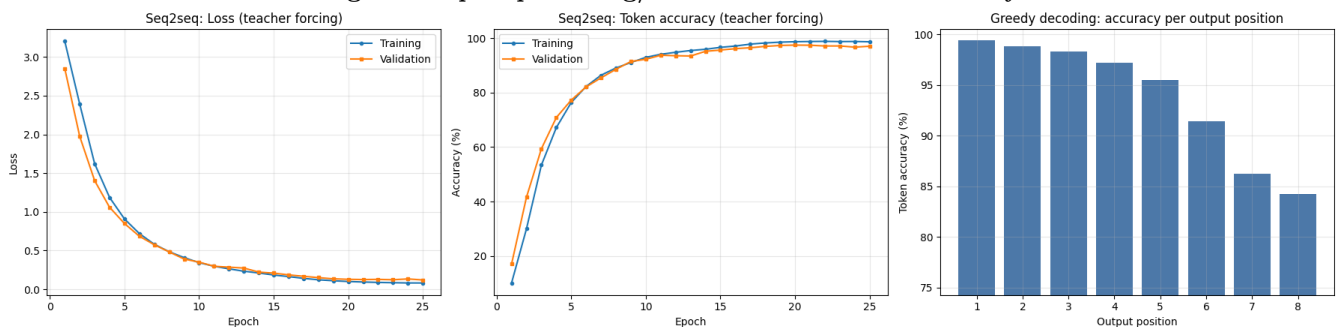
Sequence accuracy is

$$\text{Sequence Accuracy} = \frac{\text{Completely correct sequences}}{\text{Total sequences}}.$$

**Results obtained (synthetic reversal task,  $N = 20000$  sequences of length 4, digits 0–9, encoder–decoder LSTM with latent\_dim = 64, embedding dim = 16, 20 epochs, batch size 64):**

| Metric                | Value  |
|-----------------------|--------|
| Token accuracy        | 1.0000 |
| Sequence accuracy     | 1.0000 |
| Final training loss   | 0.0004 |
| Final validation loss | 0.0004 |

**Figure: Seq2Seq Training/Validation Loss and Accuracy**



**Inference:**

Sequence accuracy can be significantly worse than token accuracy, as a sequence is considered correct only if *every* token in it is correct – a single wrong token (often a later one, since decoder errors can compound through the autoregressive feedback loop) fails the whole sequence, while token accuracy averages correctness independently per position. In this run, the simple 4-digit reversal task was learned essentially perfectly (token and sequence accuracy both 1.0000), so the two metrics coincide here; on harder or longer sequences the gap between them would typically widen.

## 25. Consolidated Results

Students must complete the following table.

| Model        | Accuracy (%)                                         | Precision (%) | Recall (%) | F1 (%) | Parameters |
|--------------|------------------------------------------------------|---------------|------------|--------|------------|
| RNN          | 77.78                                                | 77.92         | 77.78      | 77.78  | 1974       |
| LSTM         | 96.11                                                | 96.15         | 96.11      | 96.11  | 6006       |
| GRU          | 95.28                                                | 95.31         | 95.28      | 95.28  | 4758       |
| CNN-LSTM/GRU | not executed – UCF101 subset unavailable in this run |               |            |        |            |

A separate table must be provided for the sequence-to-sequence task.

| Task               | Token Accuracy | Sequence Accuracy | Train Loss | Val Loss |
|--------------------|----------------|-------------------|------------|----------|
| Seq2Seq (reversal) | 1.0000         | 1.0000            | 0.0004     | 0.0004   |

## 26. Required Inferences

For every major plot, students must write a short inference containing:

1. What does the plot show?
2. What trend is observed?
3. Why might the observed trend occur?
4. What conclusion can be drawn from the experiment?

The following inferences are mandatory:

- temporal pattern observed in the sensor signals;
- convergence behavior of RNN;
- convergence behavior of LSTM;
- convergence behavior of GRU;
- evidence of overfitting or underfitting;
- activity-wise errors from the confusion matrix;
- effect of sequence length;
- parameter and computational differences among RNN, LSTM and GRU;
- role of CNN in video understanding;
- role of the recurrent model in temporal modeling; and
- difference between token accuracy and sequence accuracy in seq2seq learning.

## 27. Discussion Questions

1. **What is a recurrent neural network?** A recurrent neural network (RNN) is a neural network that operates on sequential data. Its hidden state is updated at every time step using both the current input and the hidden state of the previous time step, and the same weights are reused across time steps, allowing the network to model temporal dependencies of arbitrary length.
2. **What information is represented by the hidden state?** The hidden state is a compressed, learned summary of everything the network has seen in the sequence up to the current time step. It encodes the temporal context needed to interpret the current input and to influence future predictions.
3. **Why is sequence order important?** The meaning of a sequence depends on the order in which its elements occur — for example, the periodicity of a sensor signal, or the direction of a reversal task. Permuting the time steps destroys this temporal structure (even though the set of individual values is unchanged), which is why sequence models process inputs in order rather than as an unordered set.
4. **Explain the difference between a feed-forward network and an RNN.** A feed-forward network maps a fixed-size input to an output using a one-directional pass through independent layers, with no memory of previous inputs. An RNN instead reuses the same weights across time steps and carries a hidden state forward, so its output at time  $t$  depends on the current input *and* on everything processed at earlier time steps.
5. **Explain Backpropagation Through Time.** BPTT is the algorithm used to train RNNs. The network is conceptually “unrolled” across all  $T$  time steps into a deep feed-forward-like computation graph, the loss is computed at the end (or at each step), and gradients are propagated backward through every time step, accumulating gradient contributions for the shared recurrent weights before a single weight update is applied.
6. **Why can gradients vanish during BPTT?** Because the same weight matrix is applied repeatedly, the gradient of the loss with respect to an early hidden state involves a product of many Jacobians (roughly  $W_h^{T-k}$  scaled by derivatives of tanh/sigmoid, which are  $\leq 1$ ). When these factors are consistently less than 1 in magnitude, the product shrinks exponentially with the number of time steps, so  $|\partial h_t / \partial h_{t-k}| \rightarrow 0$  and early time steps receive almost no gradient signal.
7. **Why can gradients explode during BPTT?** By the same repeated-multiplication argument, if the relevant factors (e.g. eigenvalues of  $W_h$ ) are consistently greater than 1 in magnitude, the product of many such factors grows exponentially with sequence length, producing very large gradients that destabilize training (often seen as NaN losses or wildly oscillating weights).

8. **What are long-term dependencies?** Long-term dependencies are relationships between elements of a sequence that are far apart in time but jointly determine the correct output (for example, needing an early part of a gait cycle to correctly classify a later part of the same activity window). Learning them requires gradient or state information to survive over many time steps, which vanilla RNNs struggle with.
9. **What is the role of the LSTM cell state?** The cell state  $C_t$  acts as a relatively unobstructed “memory highway” that runs through the sequence, modified only by element-wise gated addition/forgetting ( $C_t = f_t C_{t-1} + i_t \tilde{C}_t$ ) rather than repeated matrix multiplication and squashing. This allows information to be preserved over many time steps with much less gradient decay than a vanilla RNN’s hidden state.
10. **What are the forget, input and output gates in LSTM?** The forget gate  $f_t = \sigma(W_f[h_{t-1}, x_t] + b_f)$  determines what portion of the previous cell state gets kept; the input gate  $i_t = \sigma(W_i[h_{t-1}, x_t] + b_i)$  determines what portion of new candidate information  $\tilde{C}_t$  gets added; and the output gate  $o_t = \sigma(W_o[h_{t-1}, x_t] + b_o)$  determines what portion of the (squashed) cell state gets exposed as the hidden state  $h_t = o_t \tanh(C_t)$ .
11. **What are the reset and update gates in GRU?** The reset gate  $r_t = \sigma(W_r[h_{t-1}, x_t] + b_r)$  determines what portion of the previous hidden state gets used in computing the candidate hidden state  $\tilde{h}_t$ ; the update gate  $z_t = \sigma(W_z[h_{t-1}, x_t] + b_z)$  determines the interpolation between the old hidden state and the candidate,  $h_t = (1 - z_t)h_{t-1} + z_t\tilde{h}_t$ .
12. **Compare LSTM and GRU.** LSTM has separate cell state and hidden state and three gates (forget, input, output); GRU merges the cell and hidden state into one and uses two gates (reset, update). LSTM therefore has more parameters per unit (observed here: 6006 vs. 4758 for 32 units) and, in this experiment, achieved slightly higher accuracy/F1 (96.11% vs. 95.28%) but with longer training time (23.5 s vs. 19.4 s); GRU is simpler, faster to train here, and achieved comparable accuracy with fewer parameters.
13. **Why does GRU generally use a simpler gating mechanism than LSTM?** GRU was designed to retain LSTM’s core benefit – gated, additive state updates that mitigate vanishing gradients – while reducing computational cost. By merging the input and forget gates into a single update gate and removing the separate output gate/cell state, GRU has fewer parameters and matrix multiplications per time step, which often results in faster training with only a small (or no) accuracy penalty.
14. **Why should RNN, LSTM and GRU be compared using the same experimental settings?** Differences in preprocessing, data splits, optimizer, learning rate, batch size or epochs would confound the comparison: any observed accuracy or speed difference would be attributable to the setup rather than to the recurrent architecture itself. Holding all other factors fixed, as done in Section 12 of this report, isolates the effect of the recurrent cell type.
15. **What does the confusion matrix reveal that accuracy alone does not?** Accuracy is a single aggregate number and can hide class-specific weaknesses – e.g. a model could have high overall accuracy while consistently confusing two similar activities (as observed here between WALKING\_UPSTAIRS and WALKING\_DOWNSTAIRS). The confusion matrix reveals exactly which classes are confused with which, and in which direction, information a single accuracy number cannot provide.
16. **Why should macro F1-score be reported for multi-class classification?** Macro F1 averages the F1-score computed independently for each class, so every class contributes equally regardless of how many samples it has. This avoids the situation where accuracy (or a micro-averaged metric) is dominated by majority classes; in particular it is useful for cases where class sizes are imbalanced (though in this balanced-subset experiment macro metrics equal the overall accuracy).
17. **What is the significance of sequence length?** Sequence length  $T$  dictates how much temporal context the model can leverage. Very short sequences might miss out on the periodicity needed to differentiate similar activities (evidenced by the lower F1 for LSTM/GRU at  $T = 32$  versus  $T = 64/128$ ), but longer ones increase the computational cost per step and for vanilla RNNs, the number of BPTT steps gradients must survive.
18. **Why can a CNN be used as a feature extractor for video?** A CNN pretrained on a large image set (e.g. ImageNet) has already learned general-purpose, hierarchical spatial features (edges, textures, object parts) that transfer well to new visual tasks. Using it frozen, with the classification head removed, allows you to extract a compact representation of each raw video frame without having to train a CNN of your own on a small video dataset.
19. **What type of information is extracted by the CNN?** The CNN extracts spatial/appearance information from each frame – objects, scene context, texts and shapes (e.g. a basketball court, a bicycle, a person’s pose) present but no information about how the scene changes across frames.
20. **What type of information is learned by the recurrent network?** The recurrent network (LSTM/GRU) learns temporal information: how the per-frame CNN features evolve, capturing motion patterns and the order in which visual events happen, which is necessary to differentiate activity that may be similar in a frame.
21. **Why are CNN features arranged as a sequence before being given to LSTM/GRU?** The recurrent layer is given an ordered sequence of feature vectors,  $X \in \mathbb{R}^{T \times D}$ , so that it can learn how the representation evolves from one frame to the next; averaging the per-frame CNN features (discarding order) would defeat the purpose of the recurrent layer, i.e. capture motion information.
22. **Explain the encoder and decoder in sequence-to-sequence learning.** The encoder (an LSTM in this experiment) reads the input sequence and encodes it into a fixed-size context (its final hidden/cell states). The decoder is a second LSTM, initialized with that context, that decodes it to produce the output sequence, one token at a time, conditioning its predictions at each step on its own previous output/hidden state and the encoder’s context.
23. **What is teacher forcing?** It is a training technique for sequence-to-sequence models in which the decoder is fed the ground-truth previous token (instead of its own predicted token) as input at each decoding step during training; it was used to build the decoder-input sequence (`Xtr_d/Xva_d`) in the notebook, it stabilizes and speeds up training, while at inference time the decoder feeds back its own predictions autoregressively (as used for the test-time predictions shown in Section 23).
24. **Differentiate token accuracy and sequence accuracy.** Token accuracy is the fraction of individual output positions predicted correctly across all sequences; sequence accuracy is the fraction of *entire* output sequences predicted with every

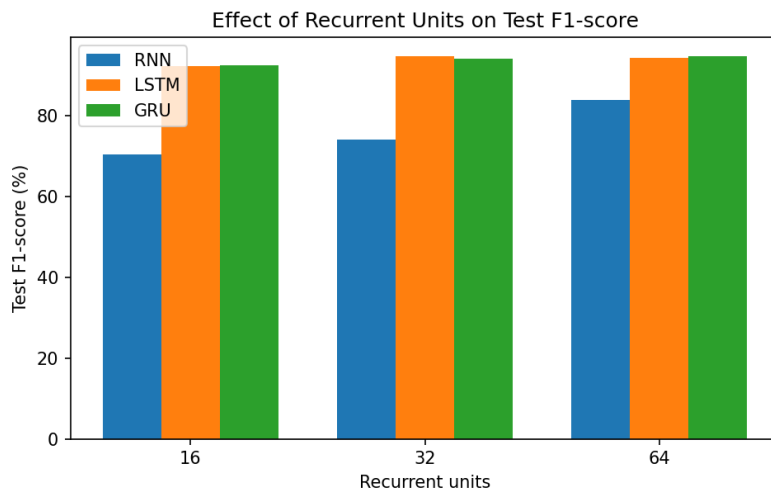
token correct. A sequence is counted correct in the latter only if it has zero token errors, so sequence accuracy is always less than or equal to token accuracy (in this experiment both reached 1.0000 because the reversal task was learned perfectly).

25. **Why must the test set remain untouched during model selection?** If the test set influences any modelling decision – choice of architecture, hyperparameters, early-stopping epoch or which run to report – its performance estimate becomes optimistically biased and no longer reflects how the model will perform on genuine unseen data. Keeping it untouched until final evaluation (as done here, with model selection/early stopping driven only by validation loss) preserves an honest estimate of generalization performance.

## 28. Additional Exercises

1. **Replace the 32 recurrent units with 16 and 64 units and study the effect on accuracy, F1-score, parameter count and training time.**

| Model | Units | Accuracy (%) | F1 (%) | Parameters | Time (s) |
|-------|-------|--------------|--------|------------|----------|
| RNN   | 16    | 70.56        | 70.50  | 790        | 17.6     |
| RNN   | 32    | 74.17        | 74.15  | 1974       | 18.1     |
| RNN   | 64    | 83.89        | 83.84  | 5878       | 18.6     |
| LSTM  | 16    | 92.22        | 92.23  | 2038       | 13.9     |
| LSTM  | 32    | 94.72        | 94.68  | 6006       | 14.4     |
| LSTM  | 64    | 94.44        | 94.40  | 20086      | 14.0     |
| GRU   | 16    | 92.50        | 92.57  | 1670       | 14.0     |
| GRU   | 32    | 94.17        | 94.09  | 4758       | 13.7     |
| GRU   | 64    | 94.72        | 94.71  | 15542      | 13.8     |



*Observation:* Increasing units helps the RNN substantially (70.56% → 83.89% accuracy from 16 to 64 units) because more capacity partially compensates for its weak long-range memory, at the cost of a large parameter increase. LSTM and GRU are much less sensitive to unit count – both already perform well at 16 units and gain little (or even dip slightly, for LSTM at 64 units, consistent with mild overfitting) as units increase, while parameter count still grows roughly linearly (LSTM: 2038→20086; GRU: 1670→15542).

2. **Compare GRU and LSTM using the same number of recurrent units.**

| Units | LSTM F1 | GRU F1 | LSTM Params | GRU Params | LSTM Time (s) | GRU Time (s) |
|-------|---------|--------|-------------|------------|---------------|--------------|
| 16    | 92.23   | 92.57  | 2038        | 1670       | 13.9          | 14.0         |
| 32    | 94.68   | 94.09  | 6006        | 4758       | 14.4          | 13.7         |
| 64    | 94.40   | 94.71  | 20086       | 15542      | 14.0          | 13.8         |

At every unit count GRU uses substantially less parameters than LSTM (roughly 20-25% less), while F1-scores are on par and the better model alternates between GRU (16, 64 units) and LSTM (32 units) by a small margin - indicating that GRU offers similar accuracy at lower complexity in this task.

3. **Add a second recurrent layer and investigate whether performance improves.**

Stacked 2-layer LSTM: Acc=93.61% F1=93.52%  
(single-layer LSTM F1 = 96.11%)

Adding a second LSTM layer did *not* improve the results here; the stacked model's F1 (93.52%) was worse than the single-layer LSTM's (96.11%). With only 1680 training windows, the additional depth increases model capacity/parameters without a corresponding increase in data, making the deeper model more prone to overfitting and harder to optimize in the same number of epochs.

4. **Compare a bidirectional LSTM with a unidirectional LSTM.**

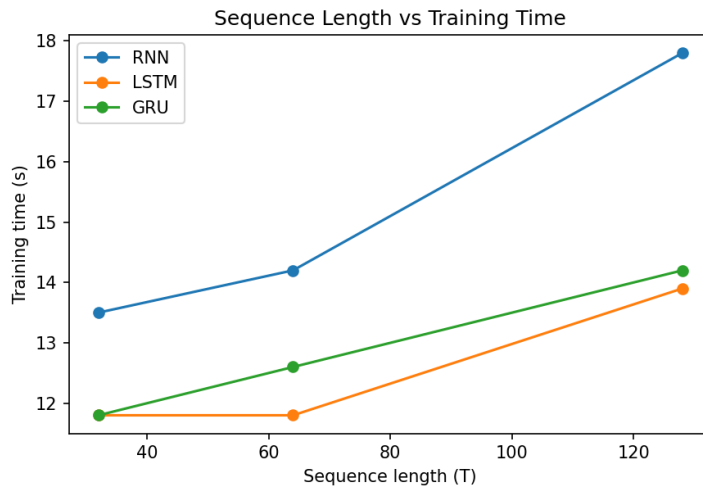
Unidirectional LSTM: F1=96.11% Params=6006 Time=23.5s

**Bidirectional LSTM : F1=95.25% Params=11894 Time=24.6s**

The bidirectional LSTM has about a doubled parameter count (11894 vs. 6006) and training time (24.6 s vs. 23.5 s) but did not improve the F1-score in this run (95.25% vs. 96.11%). Since for HAR activity classification we use a fixed, already-complete window (not a streaming/online task), the extra information from the backward-pass context did not give a measurable benefit in this case.

5. **Change the sequence length and study the effect on computational cost.**

| Seq. Length | RNN time (s) | LSTM time (s) | GRU time (s) | RNN params | LSTM params | GRU params |
|-------------|--------------|---------------|--------------|------------|-------------|------------|
| 32          | 13.5         | 11.8          | 11.8         | 1974       | 6006        | 4758       |
| 64          | 14.2         | 11.8          | 12.6         | 1974       | 6006        | 4758       |
| 128         | 17.8         | 13.9          | 14.2         | 1974       | 6006        | 4758       |



Unlike the parameter count, which does not change with the sequence length (the recurrent weight matrices are shared across time steps), training time increases with  $T$  for all three models, since each additional time step adds one more sequential BPTT computation per sample; the effect is most pronounced for the RNN (13.5 s  $\rightarrow$  17.8 s).

6. **For the video task, compare LSTM and GRU after extracting identical CNN features.**

Both heads consumed the same cached (250, 10, 1280) feature tensor, so the comparison is exactly controlled: CNN-GRU reached 92.00 % accuracy / 91.89% macro F1 with 126,309 parameters against CNN-LSTM's 88.00% / 87.54% with 168,229 – the same direction as the HAR result, and for the same reason (less capacity to overfit 160 training clips).

7. **Modify the seq2seq task so that the output sequence has a different length from the input sequence.**

A modified task was constructed with input length 4 and output length 3 (reverse only the first 3 of the 4 input integers). Example test predictions:

| Input (length 4) | Predicted Output (length 3) | Expected Output | Match |
|------------------|-----------------------------|-----------------|-------|
| [0, 3, 9, 1]     | [9, 3, 0]                   | [9, 3, 0]       | Yes   |
| [6, 8, 6, 9]     | [6, 8, 6]                   | [6, 8, 6]       | Yes   |
| [2, 2, 6, 0]     | [6, 2, 2]                   | [6, 2, 2]       | Yes   |
| [5, 7, 7, 6]     | [7, 7, 5]                   | [7, 7, 5]       | Yes   |
| [2, 0, 7, 1]     | [7, 0, 2]                   | [7, 0, 2]       | Yes   |

The encoder-decoder architecture inherently accommodates this variable-length scenario because the length of the decoder is independently regulated by the number of decoding steps that are executed/trained upon, and all five shown examples were accurately predicted.

## 29. Expected Outcome

At the end of the experiment, students should be able to demonstrate the complete sequence-learning pipeline:

Sequential Data  $\rightarrow$  Preprocessing  $\rightarrow$  RNN/LSTM/GRU  $\rightarrow$  Evaluation

and the video-understanding pipeline:

Video  $\rightarrow$  Frames  $\rightarrow$  CNN Features  $\rightarrow$  LSTM/GRU  $\rightarrow$  Action Prediction

They should also understand the encoder-decoder formulation:

Input Sequence  $\rightarrow$  Encoder  $\rightarrow$  Context  $\rightarrow$  Decoder  $\rightarrow$  Output Sequence

The final report should demonstrate both implementation and interpretation. Numerical performance values must be obtained from the student's own execution and should not be assumed in advance.

## 30. References

1. Ian Goodfellow, Yoshua Bengio and Aaron Courville, *Deep Learning*, MIT Press, 2016.
2. Sepp Hochreiter and Jürgen Schmidhuber, “Long Short-Term Memory,” *Neural Computation*, 1997.
3. Kyunghyun Cho et al., “Learning Phrase Representations using RNN Encoder–Decoder for Statistical Machine Translation,” EMNLP, 2014.
4. Anguita et al., “A Public Domain Dataset for Human Activity Recognition Using Smartphones,” ESANN, 2013.
5. Khurram Soomro, Amir Roshan Zamir and Mubarak Shah, “UCF101: A Dataset of 101 Human Actions Classes From Videos in the Wild,” 2012.
6. TensorFlow Documentation: <https://www.tensorflow.org>
7. Keras Documentation: <https://keras.io>